



University of Southern Brittany
Vannes, France

Project Report
on
Lidar Point Land Cover Classification Based on
Machine Learning and Distributed Spark
Framework

Big Data Course
Copernicus Master's in Digital Earth
(GeoData Science Track)

Group Members:

Rabina Twayana
Sahar Mohamed
Dastan Nurbukey

Submitted To:

Prof/ Frederic Raimbault
(IRISA – UBS)

16 November 2024

ABSTRACT

This experiment introduces a reproducible PySpark pipeline designed for end-to-end processing and classification of large point-cloud datasets. The workflow integrates exploratory data analysis, systematic preprocessing, and feature engineering to create informative features from raw point clouds. Model training is accomplished with a Random Forest classifier, optimized via grid search and cross-validation, and evaluated using standard metrics. To examine scalability, the study measures runtime across different Spark cluster sizes and sampling fractions, providing speedup curves that illustrate the balance between parallelism and orchestration costs. Comprehensive code, parameter configurations, and processing recipes are supplied to facilitate reproducibility. The results reveal the impact of dataset size and sampling approach on both classification accuracy and system efficiency, offering actionable strategies for distributed point-cloud analytics in Spark.

Keywords: PySpark, point-cloud, feature engineering, Random Forest, scalability, speedup curves, reproducibility

TABLE OF CONTENTS

<i>ABSTRACT</i>	<i>i</i>
<i>1 INTRODUCTION</i>	<i>1</i>
1.1 Background	1
1.2 Objectives	1
1.3 FRACTAL Datasets.....	1
<i>2 TOOLS</i>	<i>2</i>
<i>3 METHODOLOGY</i>	<i>2</i>
3.1 Preprocessing and Feature Engineering.....	2
3.2 Model Training and Evaluation	3
3.3 Model Inference	4
<i>4 TECHNICAL CHOICES</i>	<i>4</i>
<i>5 EXPERIMENT SETUP</i>	<i>5</i>
<i>6 RESULTS</i>	<i>6</i>
6.1 Exploratory Data Analysis	6
6.2 Hyperparameter Tuning	7
6.3 Model Evaluation.....	7
6.4 Experimental Findings	7
6.5 Model Inference	11
<i>7 DISCUSSION</i>	<i>11</i>
<i>8 CHALLENGES AND SOLUTIONS</i>	<i>11</i>
<i>9 CONCLUSION</i>	<i>12</i>
<i>REFERENCES</i>	<i>13</i>

1 INTRODUCTION

1.1 Background

In recent years, Airborne LiDAR (Light Detection and Ranging) or aerial laser scanning (ALS) has been a widely used technology for high-density and high-accuracy topographic data acquisition (Singh & Sreevalsan-Nair, 2021). It has been extensively adopted for areas such as building extraction, 3D urban modelling, DEM (Digital Elevation Model) generation, land cover classification, glacier monitoring, forest management, and other above-ground structures (Liu & Zhang, 2008). LiDAR captures thousands to millions of three-dimensional points per scan, producing a dense 3D point cloud. As a result, it leads to a significant increase in the data volume, imposing challenges in terms of data storage, processing, and manipulation (Gaydon et al., 2024). This is where computing in distributed systems comes into play to scale the processing of large-scale data by enabling parallel execution of tasks across multiple machines to reduce the execution time significantly. One of the most popular frameworks for big data processing and analytics is Apache Spark (Chebbi et al., 2021).

Integration of tools like Apache Spark with cloud-based big data platforms such as Amazon Elastic MapReduce (EMR) allows for running large-scale distributed data processing tasks and machine learning applications efficiently. In addition to this, Amazon Simple Storage Service (Amazon S3) buckets allow users to store and retrieve massive amounts of data. EMR and S3 integrate seamlessly, which makes the big data analysis process more flexible and efficient. Furthermore, its built-in machine learning library, MLlib, facilitates the development of scalable and efficient machine learning workflows.

1.2 Objectives

The main objective of this project is to experiment with the efficiency and scalability of Machine Learning (ML) algorithms in the distributed computing environment using PySpark for the land cover classification on the large FRACTAL dataset.

The secondary objectives of this project are as follows:

- To perform Exploratory Data Analysis (EDA) on large-scale datasets.
- To develop an efficient and reproducible ML pipeline that includes preprocessing, training, and evaluation of the model.
- To access the scalability through speedup curves by comparing execution time on varying node sizes (particularly executors) for different fractions of the datasets.

1.3 FRACTAL Datasets

FRACTAL stands for FRENch ALS Clouds from TArgeted Landscapes. This is a large and open 3D point cloud benchmark dataset for the semantic segmentation of diverse landscapes. The High-density Aerial Lidar Scanning (ALS) survey was conducted in five spatial domains in Southern France, covering 17,280 km² (an average of 3456 km²), and at least 100 km away from one another (Gaydon et al., 2024).

FRACTAL datasets consist of a total of 100000 patches and is split into a train-test validation set at an 80%-10%-10% ratio, that is, 80000 to the train set, 10000 to the validation set, and 10000 to the test set. Point clouds (37 pts/m² on average) are extracted from sampled 50 × 50 m geometric patches and colorized using near-infrared, red, green, and blue channels from ORTHO HR aerial images, aligned vertically with the points. Finally, the point clouds were annotated with seven

semantic classes: ground, vegetation, building, water, bridge, permanent structure, and other. Points are colorized with high-resolution aerial imagery, and each data patch includes detailed metadata on class distribution and landscape features (Gaydon et al., 2024).

The FRACTAL data was sampled based on a balanced sampling strategy to enhance the representation of rare classes (such as water, bridge, and permanent structures), challenging landscapes (mountainous areas and seashores), and complex urban scenes. Since this dataset is diverse in spatial extent and landscape types, it is well-suited for the evaluation of point cloud classification methods (Gaydon et al., 2024).

2 TOOLS

The tools that are used in this project are briefly described below:

a. Apache Spark

Apache Spark is the most widely used engine for scalable computing. It is a unified engine for large-scale data processing, analytics, and machine learning algorithms on single-node machines or clusters (The Apache Software Foundation, n.d.). The framework is based on the MapReduce paradigm, and it has numerous APIs implemented in Python, Java, and Scala (Polak, 2023).

b. PySpark

PySpark is the Python API for Apache Spark that allows access to the Spark distributed environment using Python. This allows users to build scalable data pipelines and machine learning workflows using familiar Python syntax. Within a PySpark application, *SparkContext* has a parameter called *_gateway*, which manages the connection interface to pass the Py4J application to the Spark Server running on JVM (Polak, 2023).

c. AWS- EMR (Amazon Web Service-Elastic MapReduce)

Amazon EMR is a big data processing service that provides tools and workflows to manage massive data in the cloud. With Amazon EMR, users can get access to web web-based big data platform, such as an S3 bucket, to process the massive amount of data using the available open-source tools like Trino, Apache Spark, and Apache Hive (Amazon Web Services, n.d.).

d. Amazon S3 (Simple Storage Service)

S3 is an industry-leading scalable storage service based on object storage technology that provides high security, durability, and low latency. The basic storage units of Amazon S3 are objects, which are organized into buckets. Each object is identified by a unique and metadata can be used to flexibly organize the data. Data can be accessed easily through the Amazon Console or the powerful S3 API via an internet connection (Cloudian, n.d.).

3 METHODOLOGY

3.1 Preprocessing and Feature Engineering

The methodology adopted in this study follows a structured experimental workflow designed to evaluate the scalability of a distributed Random Forest classifier implemented using Apache Spark. The pipeline comprises a sequence of deterministic steps that ensure reproducibility, controlled data loading, preprocessing, feature engineering, model training, evaluation of the model in test datasets, and finally plotting the speedup curve for our experiments. First, the dataset is ingested from S3 storage using Spark's S3A connector. To mitigate the overhead associated

with numerous small Parquet files and avoid loading all the data, then taking the fraction from it, a sample data loading strategy is employed. Instead of sampling rows within files, a fixed proportion of input files was selected directly from S3 and loaded directly to the driver manager. This approach enables predictable I/O cost and consistent runtime behavior across experimental runs. After ingestion, a set of lightweight feature engineering transformations is applied. These include:

- a. **Coordinate decomposition**, where the original 3-D xyz vector is split into its X, Y, and Z components;
- b. **Class remapping**, which combines child classes of the same parent class, like vegetation into one class, so instead of having 11 classes, this approaches ensure only 9 classes (Gaydon et al., 2024).
- c. **NDVI (Normalized Difference Vegetation Index)**: spectral indices to show the greenness and drought ratios for the classes
- d. **VectorAssembler**: assemble features into a feature vector column to feed it to the Random forest classifier to be trained on it, besides the label column “Classification” column to be used as a true label for training, and evaluate the model.

All transformations are implemented as Spark Transformer classes to ensure compatibility with Spark’s pipeline abstraction and to maintain distributed execution. The processed features are then combined using VectorAssembler, and the resulting feature vector is used to train a Random Forest classifier using Spark MLlib.

3.2 Model Training and Evaluation

In modeling, a hyperparameter of the RandomForest classifier was tuned first via Cross Validator (CV) and grid search on the 1% subset. The best parameters obtained from this experiment were used to train the models for the other experiment, as CV-based model training is time-consuming. Evaluation involves calculating accuracy on both training and test sets, along with collecting Spark stage and task metrics into log files. For experiment orchestration, logs were saved for each data fraction (1%, 3%, 5%, 10%). Each fraction undergoes testing across executor counts of 8, 16, 24, and 30, which are then used to generate speedup curves. Speedup experiments are designed by running the full pipeline across the specified dataset fractions and executor counts. The pipeline is executed for multiple combinations of dataset size fractions and executor counts. For each configuration, performance metrics, including total runtime, model training time, accuracy, entire workflow time, and Spark stage/task-level statistics, were recorded. Metric collection encompasses:

- Functional metrics such as accuracy (for both training and testing) and feature importances derived from the RandomForest model.
- Performance metrics, including total runtime, training duration, stage and task metrics (like shuffle read/write sizes, executor CPU time), and the number of partitions.
- Reproducibility artifacts consisting of result logs for each run that document all metrics, stored within Spark event log files.

3.3 Model Inference

During the experiment, the fitted preprocessing pipeline and trained model were exported and saved in S3 storage. Then, the saved trained model was loaded and made a prediction on the test dataset. From this analysis, the model was able to predict the classes based on the features that were passed. The result was saved in the inference log file during the test.

4 TECHNICAL CHOICES

The project's technical design blends well-founded decisions from Apache Spark MLlib with practical engineering choices in its pipeline structure. These decisions are intended to deliver a scalable and reproducible process for feature engineering and Random Forest model training on a large, file-partitioned point-cloud dataset. The main choices are outlined below by theme, with each one discussed in relation to its impact on performance, correctness, and reproducibility:

a. API and programming model: This pipeline utilizes Spark's DataFrame-based API alongside Spark MLlib's high-level abstractions (for example: *Transformer*, *Pipeline*, *VectorAssembler*, *RandomForestClassifier*). The DataFrame API is favored for its optimizer-driven execution, seamless integration with Spark SQL features like Adaptive Query Execution, and its simpler composition of distributed transformations, when compared to the RDD-based API. By implementing preprocessing as Spark Transformer classes, distributed execution semantics are maintained, and transformations can be fitted, stored, and reused as a single pipeline artifact, supporting reproducibility and deployment.

b. Data ingestion and sampling strategy were selected as the primary data reduction mechanism and avoid overloading into memory because our colleague noticed that when we used (`df.sample`), all the data from train, validation, and test were being loaded, then the fractions came as a second step, which is time-consuming. So, to overcome this issue, files were sampled first instead of data from the S3 bucket. This ensures the reduction in the number of tasks launched and provides more predictable runtime characteristics.

c. Feature engineering and representation: Feature engineering is selected based on the importance and relevance to the classification label, which has the classes. The pipeline decomposes the xyz coordinate into X, Y, and Z and selects spectral channels (Red, Green, Blue, Infrared) together with Intensity. Besides, Spectral indices like NDVI and brightness were calculated from the spectral channels to be added to the feature extraction pipeline. These features are assembled into a dense feature vector with *VectorAssembler* before model training.

d. Model choice and evaluation: MLlib's Random Forest classifier was chosen for its resilience to feature scaling, its explainability through feature importances, and efficient parallel training on Spark executors. The model supports two training modes:

- a quick baseline using default RF parameters, and
- comprehensive hyperparameter tuning via grid search cross-validation with *CrossValidator* and *ParamGridBuilder*.

Evaluation relies on MLlib's *MulticlassClassificationEvaluator* using accuracy as the main metric, applied to both train and test splits. Cross-validation parallelism is tuned to ensure a balance between hyperparameter search speed and minimizing resource contention.

e. Memory, serialization, and shuffle tuning; The pipeline reduces JVM heap pressure and enhances shuffle performance by using the Kryo serializer (`org.apache.spark.serializer.KryoSerializer`) and increases executor and driver memory overhead where necessary. To prevent driver out-of-memory errors during model summarization or small collections, `spark.driver.maxResultSize` is set conservatively. Shuffle performance is further optimized by following a partitioning rule setting (`spark.sql.shuffle.partitions`) proportional to total available cores (roughly 2–4 partitions per core), and by configuring (`spark.sql.files.maxPartitionBytes`) to 256 MB, ensuring input splits aren't too small or too large. With Adaptive Query Execution (AQE) enabled, Spark coalesces partitions at runtime based on shuffle sizes, reducing the need for constant retuning of partition counts and enhancing stability for datasets of varying sizes (SparkCodeHub, n.d.).

f. Fault tolerance and straggler mitigation: Enabling speculative execution helps to mitigate the impact of straggler tasks that delay stage completion in heterogeneous clusters. Combined with AQE and proper partition sizing, it shortens long task completion times and improves the accuracy of runtime measurements needed for speedup calculations (SparkCodeHub, n.d.).

g. Deployment and resource orchestration: The implementation remains agnostic to cluster manager, configuration options support running on YARN, standalone Spark clusters, or local testing, while exposing executor memory, cores, and instance counts as command-line parameters. This design allows horizontal scaling experiments (varying number of executors) to be performed with minimal code changes.

5 EXPERIMENT SETUP

The primary goal of this experiment was to evaluate the scalability of our end-to-end Random Forest Classification pipeline implemented in PySpark. Specifically, we aimed to quantify the relationship between execution time and variations in data volume and allocated cluster resources (executors). The pipeline encompasses all necessary steps for a full machine learning workflow: data preprocessing, feature engineering, model training (Random Forest Classifier), and model testing. Crucially, this experiment was focused purely on throughput and execution time, not on model performance (accuracy or F1-score).

For the experiments, we used two different cluster configurations as shown in Table 1. For the Executors 8, 16, and 24 were run on EMR 1, while Executor 30 was run on EMR 2 for all the fraction of data.

Table 1: The configurations of the cluster environment.

Clusters	Number of Nodes	RAM	CPU
EMR 1	32	64 GB (Each)	8 vCPU (Each)
EMR 2	8	64 GB (Each)	8 vCPU (Each)

a. Static Cluster Configuration

To isolate the impact of dataset size and executor count, the following resource configurations were held constant across all tests:

Table 2: Static variables of the experiment.

Parameter	Configuration Value	Spark Property
Per-Executor Cores	2	spark.executor.cores
Executor Memory	8GB	spark.executor.memory
Driver Memory	6GB	spark.driver.memory

b. Experimental Variables (Independent Variables)

The experiment utilized a factorial design, systematically varying the following two independent variables:

- **Dataset Size (Data Volume):** The total number of records processed, simulating an increasing workload. *Tested Sizes:* 1% (800 train sample files, 100 test sample files), 5% (4000, 500), 10% (8000, 1000).
- **Number of Executors:** The total number of worker processes running on the cluster nodes, simulating increased computational power. *Tested Counts:* 8, 16, 24, 30.

c. Performance Metric (Dependent Variable)

The primary dependent variable measured was the **Total Elapsed Wall-Clock Time (seconds)**. This time captures the **end-to-end execution duration** of the entire Random Forest pipeline, starting from the PySpark application submission, encompassing environment setup, initial data loading from the source, and completing upon the final output of the model testing step. This metric is critical as it reflects the overall system throughput and the actual user-perceived performance for a full production run.

To enable deeper bottleneck analysis, we also measured the execution time for two critical sub-phases of the pipeline:

- **Data Preprocessing and Feature Engineering Time:** This segmented measurement isolates the duration of I/O-intensive operations and wide transformations (shuffles), which are often the primary bottlenecks in large-scale data processing.
- **Model Training Time:** This measurement isolates the computational cost incurred specifically by the Random Forest Classifier's iterative training process.

The collection of these segmented metrics, alongside the total wall-clock time, will be leveraged in the Discussion section to pinpoint exactly where parallelization gains are maximized and where diminishing returns originate.

6 RESULTS

6.1 Exploratory Data Analysis

Table 3 shows the distribution of the target variable (Classification) for the Train, Test, and Validation datasets, respectively. It has a total 11 classes and it highlights the high class imbalance.

Table 3: Target Variable Distribution

Classification Column	Class names	Train (%)	Validation (%)	Test (%)
1	Unclassified	0.56	0.53	0.67
2	Ground	38.97	39.10	40.49
3	Low vegetation	4.36	4.31	4.28
4	Medium vegetation	4.31	4.35	4.27
5	High vegetation	48.31	48.27	45.54
6	Building	2.80	2.80	3.34
9	Water	0.52	0.49	1.00
17	Bridge deck	0.13	0.10	0.16
64	Permanent structures	0.04	0.04	0.03
65	Artifact	0.01	0.01	0.02
66	Synthetic	0.00	0.00	0.00

6.2 Hyperparameter Tuning

For the hyperparameter tuning part. We run our experiment with 1% of the data with 24 executors, and it took around 19 minutes to find the best hyperparameter to evaluate the model. We tune the two important parameters *numTrees* [10, 20, 30] and *maxDepth* [5,10] of the RandomForest model. Based on this experiment, we obtain the best *numTrees* is 30 and *maxDepth* is 10. These parameters were used for the rest of the model experiments.

6.3 Model Evaluation

Table 4 shows the model performance (accuracy in %) on the training and test sets for the different executors and fraction of data. Training accuracy is consistently around **81–82%**, while test accuracy is slightly lower (**78–80%**).

Table 4: Model Evaluation (Accuracy)

Executors	Fraction 1		Fraction 5		Fraction 10	
	Train (%)	Test(%)	Train(%)	Test(%)	Train(%)	Test(%)
8	81.90	78.81	81.30	79.00	81.00	80.00
16	81.90	78.81	81.34	79.70	80.96	79.81
24	81.90	78.81	81.34	79.70	81.00	80.00
30	81.98	79.17	81.34	79.70	80.96	79.81

6.4 Experimental Findings

This section presents the empirical results obtained from our systematic scalability evaluation of the PySpark Random Forest Classification pipeline. Our analysis reveals distinct scalability patterns, identifies optimal resource configurations, and uncovers performance anomalies that warrant further investigation.

Table 5 summarizes the performance of the Spark-based preprocessing, model training, and end-to-end workflow for varying executor counts and data fractions. In the same pipeline, the time taken for preprocessing and model training was tracked separately. It reports execution time in minutes or seconds for each stage of the pipeline, along with timing derived from task and stage metrics. The entire workflow time and the elapsed time from the stage metrics are comparable as both are based on wall-clock time.

Table 5: Execution Time

Category	Executor	Fraction 1	Fraction 5	Fraction 10
Preprocessing time (min) wall-clock time	8	0.98	0.88	1.81
	16	2.64	1.25	1.95
	24	1	1.42	2.40
	30	1.08	1.07	1.004
Model Training time (min) wall-clock time	8	8.48	26.26	69.83
	16	5.18	15.96	34.92
	24	4.45	12.17	22.15
	30	3.62	10.73	30.67
Entire Workflow (min) wall-clock time	8	10.55	32.37	82.47
	16	7.25	22.23	41.78
	24	6.08	14.45	26.93
	30	8.38	20.66	35.28
Entire Execution Time (min) Task Metrics	8	114	582	1266
	16	90	462	1200
	24	84	510	1104
	30	102	498	534
Entire Elapsed Time (min) Stage Metrics	8	9.8	32	84
	16	6.5	20	41
	24	5.5	14	26
	30	7.8	20	35

a. Total Elapsed Execution Time

Figure 1 illustrates the total elapsed wall-clock time (from the stage metrics logs) as a function of executor count for each dataset size. The results demonstrate a general inverse relationship between executor count and execution time up to 24 executors, followed by performance degradation at 30 executors. For the smallest dataset (1% fraction), execution time decreased from 9.8 minutes at 8 executors to 5.5 minutes at 24 executors, representing a 43.9% reduction. The intermediate dataset (5% fraction) exhibited a more pronounced improvement, with execution time declining from 32 minutes to 14 minutes (56.3% reduction) over the same executor range. The largest dataset (10% fraction) showed the most substantial benefit from parallelization, decreasing from 84 minutes at 8 executors to 26 minutes at 24 executors, a 69.0% reduction in execution time.

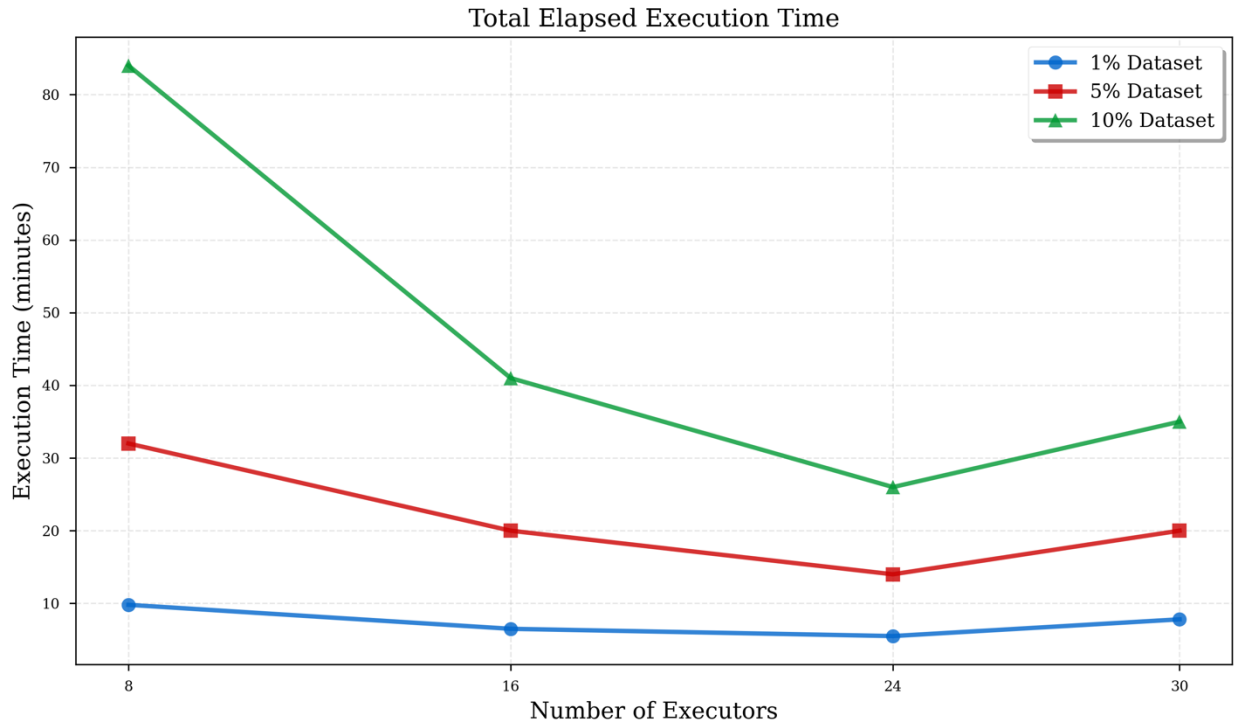


Figure 1. Total elapsed wall-clock time (minutes) as a function of executor count for three dataset sizes.

Notably, the 30-executor configuration demonstrated unexpected performance degradation across all dataset sizes. For the 10% dataset, execution time increased from 26 minutes at 24 executors to 35 minutes at 30 executors, representing a 34.6% performance penalty. This anomaly suggests that coordination overhead and inter-executor communication costs exceeded the marginal benefits of additional parallelism at this scale, consistent with theoretical predictions from Amdahl's Law regarding the serial fraction of parallel computations (Michael McCool et al., 2012).

b. Task-Level Execution Metrics

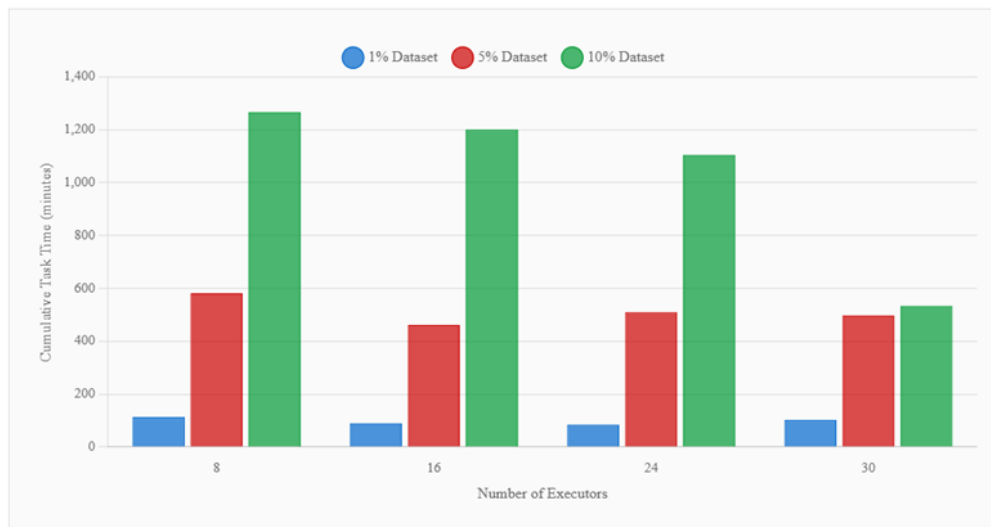


Figure 2. Cumulative task execution time (minutes) representing the sum of all parallel task durations.

Figure 2 presents the cumulative task execution time, which represents the aggregate CPU time consumed by all parallel Spark tasks throughout the pipeline execution. This metric provides insight into the degree of parallelization achieved and the efficiency of resource utilization. For the 10% dataset at 8 executors, cumulative task time reached 1,266 minutes while wall-clock time was only 84 minutes, yielding a parallelization factor of approximately 15.1x. This indicates that, on average, approximately 15 tasks were executing concurrently, demonstrating effective utilization of the available executor resources.

The 24-executor configuration maintained high task throughput with 1,104 cumulative task-minutes for the 10% dataset, while achieving the lowest wall-clock time of 26 minutes, suggesting optimal task distribution and minimal idle executor time. Conversely, the 30-executor configuration showed a dramatic reduction in task time to 534 minutes while wall-clock time increased to 35 minutes. This indicates that while individual tasks completed more quickly due to finer-grained parallelism, the overall pipeline suffered from increased coordination overhead, task serialization bottlenecks, or uneven task distribution leading to executor starvation.

c. Scalability Efficiency Analysis

To quantify the scalability characteristics of our pipeline, speedup factors relative to the 8-executor baseline configuration are computed and plotted as shown in Figure 3. For the 10% dataset, the speedup progression achieved with 8 executors (1.00x baseline), 16 executors (2.05x), 24 executors (3.23x), and 30 executors (2.40x). The sub-linear speedup pattern is consistent with [Amdahl's Law](#), which predicts that the achievable speedup is limited by the serial fraction of the computation.

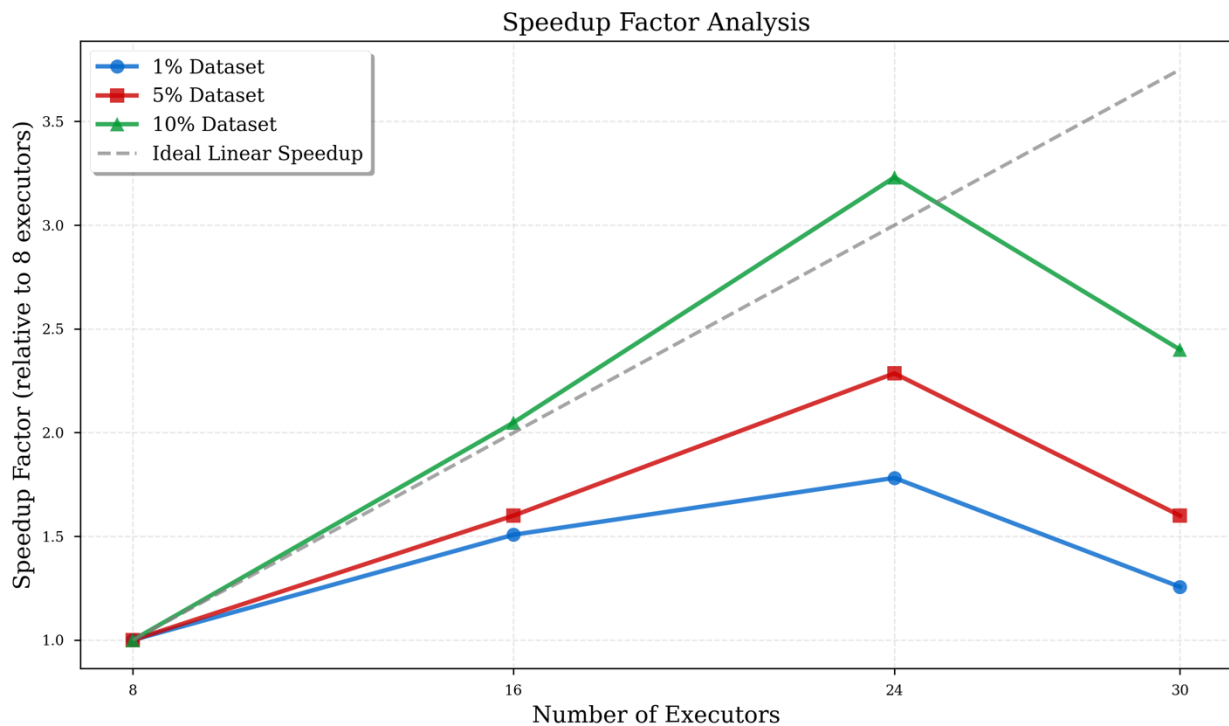


Figure 3. Speedup factor relative to 8-executor baseline for all dataset sizes.

Speedup factor (equation as mentioned below) measures how much faster a task runs on multiple processors compared to a single processor.

$$Speedup = \frac{T_1}{T_n}$$

It demonstrates strong scaling up to 24 executors across all workloads, followed by significant performance degradation at 30 executors attributable to shared cluster resource contention. Furthermore, we examined weak scaling behavior by analyzing how execution time changed when both data volume and computational resources were increased proportionally. When data increased from 1% to 10% (10x growth) for 8 executors, execution time increased by a factor of 4.73 (from 5.5 to 26 minutes). This super-linear relationship indicates that data volume has a more significant impact on execution time than computational resources can compensate for, likely due to increased shuffle operations, memory pressure, and garbage collection overhead at larger data scales. From the experiment, it can be concluded that larger datasets benefit more from parallel processing.

6.5 Model Inference

The preprocessing pipeline and trained model corresponding to 3 data fractions were loaded for the experiment from S3 storage. Loading the transformation pipeline and model took 2.011 and 3.53 second respectively. Applying the preprocessing transformation took 4.852 seconds on the whole train dataset, while prediction finished in 1.74 seconds for the first 20 rows.

7 DISCUSSION

The experimental results conclusively demonstrate that 24 executors represent the optimal configuration for workloads comparable to those tested in this study. This configuration consistently delivered the lowest execution times across all dataset sizes while maintaining high resource utilization efficiency. The performance degradation observed at 30 executors highlights the importance of empirically determining the optimal parallelism level rather than assuming that more resources invariably yield better performance. Model training emerged as the dominant bottleneck in the pipeline, consuming 95-98% of total execution time. The negligible preprocessing overhead suggests that for this particular workload, I/O operations and feature transformations do not constitute performance-limiting factors. However, this finding may not generalize to production scenarios involving cold storage systems, complex data quality operations, or substantial feature engineering logic. The discrepancy between task-level metrics and wall-clock time at high executor counts suggests that task scheduling and coordination overhead become significant bottlenecks when parallelism exceeds optimal levels.

8 CHALLENGES AND SOLUTIONS

Throughout the development and execution of the pipeline, several technical challenges emerged, each requiring specific mitigation strategies to ensure stable and effective experimentation.

a. The Small-File Problem: One of the main challenges was the large number of small Parquet files in the dataset, which caused excessive task scheduling overhead and inefficient use of cluster resources since all the data would be loaded into the driver memory. When an action was triggered, only the requested fractions were processed. To solve this, file-level sampling was implemented

so that only the necessary fractions are directly loaded without the need to load all the data and then take the queried fractions.

b. Memory Constraints During Shuffle Phases: The Random Forest training process resulted in considerable shuffle traffic, occasionally causing executor memory errors. To address this, we increased executor and driver memory overhead, enabled the Kryo serializer, and adjusted shuffle partitions, all of which reduced memory pressure. Adaptive Query Execution (AQE) further enhanced stability by dynamically coalescing shuffle partitions based on their sizes. The driver and executor's memory were set to be 6 GB and 8 GB, respectively.

c. Straggler Tasks and Long-Tail Execution: Uneven workload distribution and variations in input file sizes caused some executors to consistently lag. By enabling speculative execution, Spark could relaunch slow tasks on different executors, speeding up overall stage completion and making runtimes more consistent. This worked well for a small percentage of outliers, but when applied to larger fractions like 5% or 10%, it resulted in too many tasks being reassigned to executors. This increased the workload on those executors and introduced additional delays, as examined in the results and discussion sections.

d. Enabling Client deploy mode vs cluster deploy mode: When we enabled client mode in our eight-node cluster, it excluded the driver from the cluster, inadvertently assigning all resources to the client machine. This led to an out-of-resources error as we tried running several tasks with 30 executors, even after scaling horizontally with equivalent RAM and vCPU. The issue persisted until we switched to cluster mode, placing both the driver and executors inside the cluster, ensuring stable production operation since in cluster mode, resources are managed by the cluster manager (e.g, YARN).

e. Multi-Users problem in the 32-node cluster: Despite having a large cluster with 32 nodes, 64 GB RAM, and 8 vCPUs per node, all groups experienced queue delays when submitting their tasks. This was because each group member was running different experiments with varying configurations. The issue was resolved by recommending horizontal scaling: assigning each executor a fixed 8 GB RAM and 2 vCPUs while increasing the number of executors, but not exceeding 30. This approach ensured a balance between system resources and executor configurations.

9 CONCLUSION

In conclusion, this project presents a scalable PySpark pipeline designed for large point-cloud land-cover classification using the FRACTAL dataset. The workflow integrates exploratory analysis, structured feature engineering, deterministic sampling, and distributed Random Forest training and inference. Experimental evaluation across multiple cluster sizes and sampling fractions shows clear strong-scaling benefits up to 24 executors, after which performance declines due to scheduling and coordination overhead. Training time dominates total runtime for model fitting, while preprocessing and inference remain comparatively lightweight. Speedup analysis, combined with task and stage metrics, highlights how executor count, sampling strategy, and parallelism settings influence end-to-end performance. The results indicate that a Spark-native Random Forest pipeline can achieve both stable predictive accuracy and efficient distributed

execution on point-cloud data. The study also identifies limitations, including dependency on a single dataset and model type, and highlights opportunities for future work, such as extending the benchmark to additional algorithms or using parallel random forest models, cluster configurations, and spatial generalization analyses. Overall, the findings provide a baseline for designing performant large-scale geospatial pipelines and offer practical guidance on selecting executor counts, sampling strategies, and configuration parameters for Spark-based processing of high-volume point-cloud workloads.

REFERENCES

- Amazon Web Services, Inc. (n.d.). *Amazon EMR*. Retrieved November 16, 2025, from <https://aws.amazon.com/emr/>
- Apache Spark. (n.d.). *Spark MLlib documentation*. <https://spark.apache.org/docs/latest/ml-guide.html>
- Chebbi, I., Mellouli, N., Farah, I. R., & Lamolle, M. (2021). Big Remote Sensing Image Classification Based on Deep Learning Extraction Features and Distributed Spark Frameworks. *Big Data and Cognitive Computing*, 5(2), 21. <https://doi.org/10.3390/bdcc5020021>
- Cloudian. (n.d.). *S3 Storage: How it Works, Use Cases and Tutorial*. Cloudian Blog. Retrieved November 16, 2025, from <https://cloudian.com/blog/s3-storage-behind-the-scenes/>
- Gaydon, C., Daab, M., & Roche, F. (2024). *FRACTAL: An Ultra-Large-Scale Aerial Lidar Dataset for 3D Semantic Segmentation of Diverse Landscapes* (No. arXiv:2405.04634). arXiv. <https://doi.org/10.48550/arXiv.2405.04634>
- Liu, X., & Zhang, Z. (2008). *LIDAR DATA REDUCTION FOR EFFICIENT AND HIGH QUALITY DEM GENERATION*.
- McCool, M., Robison, A. D., & Reinders, J. (2012). *Structured Parallel Programming: Patterns for Efficient Computation* (pp. 39–75). Morgan Kaufmann.
- Polak, A. (2023). *Scaling machine learning with Spark: Distributed ML with MLlib, TensorFlow, and PyTorch*. O'Reilly.
- Singh, S., & Sreevalsan-Nair, J. (2021). A Distributed System for Optimal Scale Feature Extraction and Semantic Classification of Large-Scale Airborne LiDAR Point Clouds. In D. Goswami & T. A. Hoang (Eds.), *Distributed Computing and Internet Technology* (Vol. 12582, pp. 280–288). Springer International Publishing. https://doi.org/10.1007/978-3-030-65621-8_18
- SparkCodeHub. (n.d.). *Mastering Apache SparkConf: A comprehensive guide to configuring Spark applications*. Retrieved November 16, 2025, from <https://www.sparkcodehub.com/spark/configurations/conf>
- The Apache Software Foundation. (n.d.). *Apache Spark: Unified Engine for Large-Scale Data Analytics*. Retrieved November 13, 2025, from <https://spark.apache.org/>

AI Disclaimer: Used for rephrasing and rewording some sentences to enhance clarity and conciseness.